서투리 판정 로직 정리

Gemma4 받아쓰기와 Whisper 억양 판별이 합쳐져 사투리 비율·표준어 점수·최종 점수가 나오기까지
2026-10-13 서비스 코드 기준 · `/data/seotturi/seotturi-web`

한 줄씩 요약

- **사투리 단어**는 Gemma4가 정합니다. Gemma가 **방언 줄에서 표준어 줄로 바꾼 어절**이 곧 사투리 단어입니다. 방언 사전이나 임계값은 쓰지 않아요.
- **지역과 억양 강도**는 Whisper 억양 모델이 정합니다. 먼저 "지역 억양인가"를 보고, 그다음 "어느 지역인가"를 보는 두 단계 판정입니다.
- **사투리 비율** = (사투리 단어 + 억양 어절) ÷ 전체 어절. 억양 어절은 Gemma가 바꾸지 않은 어절 수에 억양 강도를 곱한 값입니다.
- **최종 점수** = 표준어 점수(100 − 사투리 비율) + 답변 점수(단어 선택·유창성·정보). 200점 중 140점 이상이면 합격입니다.
- 발음 검사(wav2vec2)는 화면과 코치 피드백에만 나오고 점수에는 들어가지 않습니다.

1. 구성 요소
2. 전체 흐름
3. 녹음 자르기와 전처리
4. Gemma4 받아쓰기 (어댑터 켬)
5. Whisper 억양 판별
6. 사투리 단어 판정
7. 발음 검사
8. 유창성 측정
9. 답변 단위 종합: 지역 판정과 사투리 비율
10. Gemma4 코치 피드백 (어댑터 끔)
11. 기준표 채점
12. 최종 점수

부록 A. Gemma 어댑터 교체 기준
부록 B. 알아둘 점과 한계
부록 C. 코드 위치

1. 구성 요소

판정에는 프로세스 세 개, 모델 네 가지가 관여합니다. 웹 서버(app.py)가 녹음을 받아 각 모델에 보내고, 돌아온 결과로 점수를 계산합니다.

프로세스 (포트)	모델	하는 일	현재 상태
app.py (8787)	—	녹음 수신, 전처리, 모델 호출, 사투리 비율·점수 계산	—
engine.py (8788)	Gemma 4 12B + LoRA 어댑터 켬	받아쓰기: **방언 줄**(들은 그대로)과 **표준어 줄**(바꾼 문장)	어댑터 **lora_s6c** (rank 144 결합 어댑터)
engine.py (8788)	Gemma 4 12B 기본 어댑터 끔	코치 피드백, 기준표 채점(단어 선택·정보)	—
accent.py (8790) /accent	Whisper-large-v3 인코더(고정) + 판별 헤드 2개	억양: P(서울·경기), P(강원·충청·경상·제주)	4지역 **region4** + 표준/지역 **detector_mix** 시험 macro-F1 0.812, 표준/지역 AUC 0.885
accent.py (8790) /pron	wav2vec2 XLS-R 300M 한국어 음소 인식	발음 검사: ㅚ·ㅐ·ㅞ가 ㅔ로 들리는지	표시 전용(점수 미반영)

tts.py(8789)는 코치 음성 합성용이라 판정과는 관계없습니다.

2. 전체 흐름

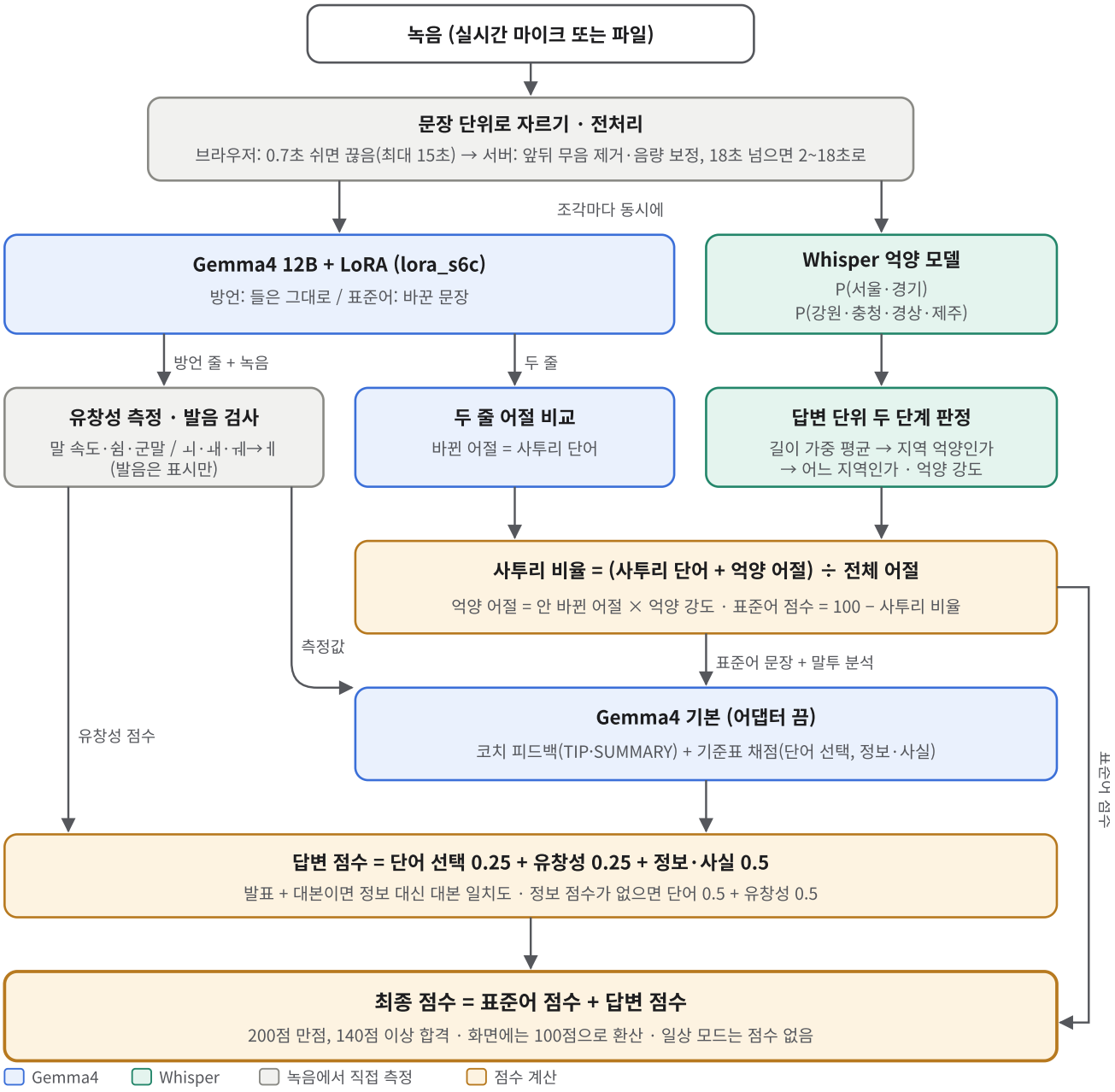


그림 1. 녹음 한 번이 점수가 되기까지. 조각 단위 처리(위 세 줄)와 답변 단위 처리(아래)로 나뉩니다.

처리 순서

1. **조각 단위**(`app.py` `transcribe`): 문장 조각마다 Gemma와 Whisper를 **동시에** 호출합니다. Gemma 결과가 나오면 방언 줄을 써서 발음 검사와 유창성 측정을 합니다. 이때 두 줄 어절 비교도 같이 해 둡니다.
2. **답변 단위**(`app.py` `coach_feedback` → `summarize_speech`): 사용자가 답변을 마치면 조각들을 모아 지역을 판정하고 사투리 비율을 계산합니다.
3. **코치**: 사투리 비율과 측정값을 프롬프트에 넣어 기본 Gemma에게 피드백을 받고, 기준표 채점을 합니다.
4. **점수**: 표준어 점수와 답변 점수를 더해 합격 여부를 냅니다.

3. 녹음 자르기와 전처리

Gemma4는 한 번에 30초까지만 들을 수 있고, LoRA는 1~20초 발화로 학습했습니다. 그래서 녹음을 문장 길이로 잘라서 보냅니다.

3.1 브라우저에서 자르기 실시간 녹음

`web/js/recorder.js`

항목	값	설명
분석 단위	20 ms 프레임	16 kHz 모노로 바꾼 뒤 프레임마다 음량(RMS)을 잽니다.
말소리 판단	음량 > max(잡음 × 2.8, 0.006)	잡음 수준은 처음 0.3초로 정하고, 조용한 프레임마다 천천히 갱신합니다.
문장 끝	0.7초 쉼	말소리가 0.7초 없으면 한 조각을 끝냅니다.
앞뒤 여유	앞 0.2초, 뒤 0.24초	말 시작 전·끝난 뒤를 조금 남깁니다.
강제 자르기	15초	15초가 넘으면 최근 4초 중 가장 조용한 곳에서 자릅니다.
버리는 조각	말소리 0.25초 미만 또는 길이 0.5초 미만	기침·잡음 같은 짧은 소리는 보내지 않습니다.

조각이 끝날 때마다 바로 서버(`/api/asr`)로 보내므로, 사용자가 말하는 동안 앞 문장의 받아쓰기가 화면에 뜹니다. 파일 업로드는 통째로 보내고 서버에서 자릅니다.

3.2 서버 전처리 모든 녹음

`server/app.py transcribe · server/analysis.py preprocess, split_audio`

- **크기 제한**: 파일 25 MB, 길이 180초.
- **앞뒤 무음 자르기**: 20 ms 프레임 음량이 기준 max(하위 15% 음량 × 3, 0.004)을 넘는 첫 프레임과 마지막 프레임을 찾고, 앞뒤로 0.3초씩 남긴 채 잘라냅니다.
- **음량 보정**: 최대 진폭이 0.5 미만이면 0.9가 되도록 키웁니다(최대 10배).
- **빈 녹음**: 말소리 프레임이 합쳐서 0.25초 미만이면 모델을 부르지 않고 "빈 녹음"으로 돌려줍니다.
- **다시 자르기**: 18초가 넘으면 2~18초 사이에서 가장 조용한 지점(0.3초 평활, 뒤쪽 선호)을 골라 자릅니다. 0.3초 미만 조각은 버립니다.

4. Gemma4 받아쓰기 (어댑터 켬)

`server/engine.py _asr, _generate_ids, _finish · gemma4-dialect/common.py`

4.1 입력과 프롬프트

조각 하나(0.3~30초, 16 kHz WAV)를 아래 프롬프트와 함께 넣습니다. 학습 때와 같은 프롬프트를 학습 코드(`common.py`)에서 그대로 가져다 씁니다.

> 다음 음성은 한국어 방언일 수 있습니다. 들은 그대로 받아쓰고, 의미와 말투를 유지한 표준어로 바꿔 주세요. 형식:
> 방언: <들은 그대로>
> 표준어: <표준어 문장>

지금 어댑터(basic 형식)는 **지역을 출력하지 않습니다**. 지역은 전부 Whisper가 정합니다.

4.2 디코딩 가드 (guard1)

장치	내용	이유
답 첫머리 강제	생성을 `방언:`으로 시작하게 붙여 둡니다.	예전 체크포인트에서 답의 5%가 엉뚱한 글자로 시작해 방언 줄을 잃었습니다.
그리디 디코딩	샘플링 없이 매번 가장 확률 높은 토큰을 고릅니다.	같은 녹음이면 항상 같은 결과.
토큰 예산	최대 토큰 = 32 + 16 × 초 (64~512)	길고 빠른 말이 잘리지 않게.
반복 중단	마지막 줄에서 20자 이하 덩어리가 8번 이상 연달아 나오면 생성을 멈춥니다.	"똥 똥 똥 …" 같은 반복 루프.
반복 보정	반복된 줄은 2번까지만 남깁니다. 방언 줄이 잘렸거나 표준어 줄이 없으면 "방언: … 표준어:"까지 주고 **표준어 줄만 다시** 생성합니다.	방언 줄은 들은 그대로 두고 변환만 다시.
프롬프트 룩업	조각이 하나일 때 입력에서 다음 토큰 후보를 찾는 가속(10토큰)을 씁니다. 엔진이 켜질 때 자체 시험에서 10% 이상 빠르고 CER이 0.02 넘게 나빠지지 않을 때만 켭니다.	속도.

반복 기준(8번)은 테스트 4,000문장 실측으로 정했습니다. 가드 없이 8번 이상 반복한 답이 v1 11개, r64 9개였고, 5번으로 낮추면 1~2개만 더 잡히는 반면 정답 문장의 0.016%도 5번 이상 반복이 들어 있었습니다.

4.3 형식 보정 (app.py)

- 방언 줄과 표준어 줄이 **둘 다 비면** 모델 출력 원문을 양쪽에 넣습니다. 두 줄이 같으므로 이 조각의 사투리 단어는 0개가 됩니다.
- **한 줄만 있으면** 그 줄을 다른 줄에 복사합니다. 이때도 사투리 단어는 0개입니다.

엔진은 사용자 요청을 모델 교체 채점보다 먼저 처리하고, 동시에 들어온 받아쓰기 요청은 최대 4개까지 묶어서 한 번에 돌립니다.

5. Whisper 억양 판별

`server/accent.py classify · server/app.py accent_probs, zone_reliable`

5.1 쓰는 조건

웹 서버는 60초마다 억양 서비스 상태(`/status`)를 확인합니다. 아래 조건을 모두 만족해야 Whisper 결과를 씁니다.

- 서비스가 준비됨(`ready`)
- 서비스 중인 모델의 시험 세트 macro-F1 ≥ 0.5 (`SEOTTURI_ZONE_MIN_F1`, 지금 0.812)

여기에 표준/지역 판별기(`std_model`)까지 있으면 **두 단계 모드**로 판정합니다. 지금 설정이 이 모드입니다. 조건을 못 맞추면 지역은 표시하지 않고, 사투리 비율은 Gemma의 사투리 단어만으로 계산합니다.

5.2 조각마다 하는 계산

Whisper-large-v3 인코더는 학습하지 않은(고정) 상태로 **한 번만** 돌리고, 그 결과를 헤드 두 개가 같이 씁니다(조각당 약 0.1초).

헤드	출력	학습 방식
detector_mix 표준/지역 판별기	q = P(서울·경기)	6지역 모델 두 개(ensemble_big, zone6_aug)의 P(서울·경기) 평균
region4 4지역 모델	지역 억양이라고 가정했을 때 P(강원), P(충청), P(경상), P(제주) (합 1)	서울 출생 화자 제외, 정답 = 녹음 세트의 지역, 녹음 환경 변형 8종으로 증강

표준 쪽 보정: $s = q \times e^{2.7}$ (서울·경기 로그 확률에 +2.7, 약 15배)
P(지역) = $1 - s \div (s + 1 - q)$
응답: 서울·경기 = 1 − P(지역), 각 지역 = P(지역) × P(그 지역 | 지역 억양)

보정 +2.7의 의미: 코치에게는 표준 화자에게 "사투리 억양이 있어요"라고 하는 실수가 더 나쁘다고 보고, 표준 쪽으로 기울였습니다. 개발 세트에서 지역 화자 조각의 90%를 잡고 서울 화자 오판을 4%로 맞춘 값입니다. 다만 시험 세트에서는 조각 단위로 지역 화자 87%를 잡고, **서울 화자 23%를 지역으로 오판**했습니다(개발 세트가 훨씬 쉬웠음).

5.3 현재 성능

지표 (시험 세트, 사투리 단어 없는 발화)	값
4지역 + 서울·경기 macro-F1	0.812
정확도	0.823
표준/지역 판별 AUC	0.885
벤치마크 세트 macro-F1	0.817

대구·경북과 부산·울산·경남은 서로 잘 구별하지 못해(F1 0.53 / 0.61) "경상" 하나로 합쳐 보여 줍니다. 전라는 학습 데이터가 없어 판별 대상이 아닙니다.

6. 사투리 단어 판정

`server/analysis.py diff`

Gemma가 낸 방언 줄과 표준어 줄을 **어절 단위로 맞대어**(difflib SequenceMatcher) 바뀐 어절을 찾습니다.

1. 두 줄을 공백으로 나누고, 각 어절에서 문장부호(. , ? ! ~ … · 따옴표 괄호 등)를 지웁니다.
2. 같은 어절은 그대로 둡니다. 바뀐 것처럼 보여도 공백·문장부호만 다르면 같은 것으로 봅니다.
3. 바뀌거나(replace) 방언 줄에만 있는(delete) 어절은 **사투리 어절**로 세고, 고침 하나("A → B")로 묶습니다. 붙어 있는 변경은 한 고침이 됩니다.
4. 표준어 줄에만 있는 단어(insert)는 바로 앞 어절에 붙여 그 어절의 고침으로 칩니다.

판정 기준이 따로 없습니다. 방언 사전(97,335개)은 단어장 화면에만 쓰이고 판정에는 쓰이지 않습니다. Gemma가 표준어로 고쳤으면 사투리, 안 고쳤으면 표준어입니다.

예시 (실제 코드 실행 결과)

방언 줄	그래가 제가 그 일을 마이 했었거든예
표준어 줄	그래서 제가 그 일을 많이 했었거든요
고침	그래가 → 그래서 · 마이 했었거든예 → 많이 했었거든요
결과	사투리 어절 3개 / 전체 6어절 (단어 비율 50%)

7. 발음 검사

`server/pron.py · server/analysis.py pron_summary`

경상 화자가 ㅞ를 ㅔ로, ㅓ를 ㅡ로 발음하는 경향을 녹음에서 직접 재는 검사입니다. 점수에는 들어가지 않고, 화면과 코치 프롬프트에만 나옵니다.

1. Gemma의 **방언 줄**을 표준 발음 규칙(연음, 비음화, 경음화 등)으로 음소열로 바꿉니다.
2. 음소 인식기(wav2vec2 XLS-R 300M, slplab 한국어 음소 모델)로 음소열을 녹음에 강제 정렬합니다(CTC).
3. 검사할 소리마다 그 구간에서 "원래 소리"와 "합쳐진 소리"의 로그 사후확률비 평균을 냅니다.
4. 이 값이 기준(서울·경기 화자의 하위 5%)보다 낮으면 "합쳐진 소리로 들림"으로 셉니다.
5. Gemma가 사투리로 고친 어절은 단어 교정에서 이미 안내하므로 빼고 셉니다.
6. 한 답변에서 걸린 횟수가 서울 기준(5%)으로는 나오기 어려울 때만(단측 이항검정 $p < 0.05$) 표시합니다.

검사	기준값	서울 대비 차이	표시
ㅚ·ㅐ·ㅞ → ㅔ	−6.44	서울 5.3%, 대구·경북 8.2%, 부산·울산·경남 11.8%	**표시**
ㅓ → ㅡ	−1.16	서울 5.0%, 다른 지역 모두 7~11% (경상 특이가 아님)	숨김
ㅆ → ㅅ	−2.57	서울 5.2%, 부산·울산·경남 3.3% (차이 없음)	숨김

8. 유창성 측정

`server/analysis.py fluency_clip, fluency_summary`

유창성은 Gemma가 매기지 않고 **녹음에서 직접** 잽니다. 음절 수와 군말은 Gemma의 방언 줄에서 셉니다.

조각마다 재는 값

- **음절**: 방언 줄의 한글 글자 수 · **군말**: "그~", "어~"처럼 물결이 붙은 한 글자
- **쉼**: 조각 안에서 0.25초 이상 이어진 무음(앞뒤 무음은 제외)
- **말한 시간**: 조각 길이 − 앞뒤 무음 − 쉼 · **구간 수**: 쉼 개수 + 1

답변 점수 (0~100)

말한 시간이 6초 미만이거나 음절이 15개 미만이면 계산하지 않습니다. 기준값은 사투리 단어가 없는 서울·경기 화자 1,195클립에서 잰 값입니다. 다른 지역 화자도 비슷하게 나와서, 이 점수는 사투리가 아니라 전달을 잽니다.

지표	계산	0점	100점	비중
말 속도	음절 ÷ 초	초당 2.0음절	4.0~5.5 (더 빠르면 2음절당 −40, 최저 40)	35%
평균 연속 길이	음절 ÷ 구간 수	3음절	12음절	25%
발성 비율	말한 시간 ÷ 전체	0.45	0.83	25%
군말	분당 횟수	20회	4회	15%

면접·발표 모드에서는 문장 사이 3~60초 침묵 횟수도 세서 코치에게 알려 줍니다(점수에는 미반영).

9. 답변 단위 종합: 지역 판정과 사투리 비율

`server/analysis.py summarize_speech, zone_verdict, accent_any_zone`

9.1 지역 판정 (두 단계)

1. 조각별 Whisper 확률을 **조각 길이로 가중 평균**합니다.
2. **지역 억양인가**: P(지역) = 1 − P(서울·경기). 0.5 미만이면 **서울·경기**로 판정합니다.
3. **어느 지역인가**: 네 지역 확률만 다시 나눠 합이 1이 되게 합니다. 1위가 0.6 미만이면 **"확실하지 않음"**, 0.6 이상이면 그 지역으로 판정합니다. 화면의 비율은 이 다시 나눈 값입니다.

첫 문장부터 판정을 보여 줍니다(예전에는 8초 이상 쌓여야 보였지만 사용자 결정으로 0초로 바꿈).

> **규칙을 고른 근거 (10/12 비교)**: 다섯 지역 중 1위 확률에 기준을 거는 방식은 답변의 25~30%를 "확실하지 않음"으로 남겼습니다. 정답 지역이 1위인데도 그랬어요(예: 지역 0.9 × 그 지역 0.65 = 0.585). 두 단계 방식은 네 문장 답변의 90%를 판정하고 85~87%를 맞힙니다. 대신 서울·경기 화자를 지역으로 보는 비율이 4~7%에서 14~17%로 늘었습니다(화자 약 100명 기준이라 대략적인 값).

9.2 사투리 비율

> **억양 어절** = Σ (조각의 전체 어절 − 사투리 어절) × 조각의 억양 강도
> · 조각의 억양 강도 = 1 − 그 조각의 P(서울·경기)
> · 답변 전체 억양 강도 ≥ 0.5 이고, 그 조각의 억양 강도도 ≥ 0.5 인 조각만 셈
> **사투리 비율** = (사투리 어절 + 억양 어절) ÷ 전체 어절
> **표준어 점수** = 100 − 사투리 비율(%)

- 억양 어절은 Gemma가 **바꾸지 않은 어절**에만 더합니다. 사투리 단어를 억양으로 한 번 더 세지 않기 위해서예요.
- 지역이 "확실하지 않음"이어도 지역 억양이라는 판정(P(지역) ≥ 0.5)만 나면 억양 어절은 셉니다.
- 억양을 어절 단위의 연속값으로 넣는 방식은 사용자 결정이며, 사투리 밀도를 연속값으로 재는 연구(Johnson et al. 2022)를 따랐습니다.

9.3 화면에 나오는 값

항목	계산
사투리 비율	(사투리 어절 + 억양 어절) ÷ 전체 어절
단어만 비율	사투리 어절 ÷ 전체 어절 (억양 제외)
억양 강도	답변 평균 1 − P(서울·경기)
지역 · 그 비율	두 단계 판정 결과와 다시 나눈 확률
사투리 교정 목록	Gemma의 고침("A → B")을 많이 나온 순서로

9.4 계산 예시 (실제 코드 실행 결과)

조각	방언 줄	어절	사투리 어절	P(서울·경기)	억양 강도	억양 어절
1	저는 그 프로젝트에서 팀장을 맡았습니다	5	0	0.20	0.80	5 × 0.80 = 4.0
2	그래가 제가 그 일을 마이 했었거든예	6	3	0.10	0.90	3 × 0.90 = 2.7
합계		**11**	**3**	**평균 0.15**	**0.85**	**6.7**

- 두 조각 모두 경상 쪽 확률이 높아 지역은 **경상(91%)**입니다.
- 단어만 비율은 3 ÷ 11 = **27%**입니다.
- 사투리 비율은 (3 + 6.7) ÷ 11 = **88%**, 표준어 점수는 **12점**입니다.
- 조각 1은 사투리 단어가 하나도 없는데, 억양 때문에 4.0어절이 사투리로 들어갔습니다.

10. Gemma4 코치 피드백 (어댑터 끔)

`server/app.py coach_feedback · server/analysis.py feedback_messages, parse_feedback`

코치는 LoRA를 끈 기본 Gemma 4 12B입니다. 녹음은 듣지 않고, 표준어 문장과 앞 단계의 측정값만 글로 받습니다. 최대 300토큰을 그리디로 생성하고 화면에 실시간으로 흘려보냅니다.

프롬프트에 들어가는 것

- 상황, 상대(페르소나), 질문, 준비한 대본(앞 1,200자), 사용자가 원하는 피드백, 말한 시간
- 답변: 조각별 **표준어 줄**에 번호를 붙인 목록
- **말투 분석**: 전체 어절, 사투리 단어 수, 억양 어절, 사투리 비율, 단어만 비율, 억양 강도, 지역 판정
- **말하기 측정**: 말 속도, 평균 연속 길이, 군말, 긴 침묵, 유창성 점수(보통 범위 같이 표시)
- **발음 측정**: 표시 조건을 통과한 경우만
- 이미 따로 안내한 사투리 교정(최대 12개)과 대본에서 빠뜨린 문장(최대 6개)
- 모드별 평가 기준: 면접(직무 연결 경험, 두괄식, 근거·결과), 발표(구조, 핵심 메시지, 연결어·전달력), 일상(자연스러운 표준어 표현)

출력 형식과 규칙

줄	내용
`TIP`	문장 번호 | 고칠 부분 인용(10자 이내) | 한 문장 조언. 최대 3개, 사투리 교정은 반복하지 않음
`SCORE`	단어 선택 | 유창성 | 정보·사실(직무·기술 정보가 없으면 -), 각 0~100. 일상 모드는 없음
`SUMMARY`	두 문장 이내 총평(잘한 점 하나, 다음에 할 일 하나)
`QUESTION`	발표 모드에서만, 청중이 할 법한 질문 하나

규칙: "너는 음성을 직접 듣지 못했다. 발음·억양은 말투 분석만 근거로 한다", "말 속도·쉼·군말은 측정값만 근거로 하고 보통 범위면 언급하지 않는다" 등이 들어갑니다.

11. 기준표 채점

`server/app.py rubric_scores · server/analysis.py RUBRICS, rubric_messages, expected_score · server/engine.py _score`

코치가 SCORE 줄에 직접 쓴 숫자는 몇 개의 둥근 값에 몰리는 경향이 있어, 단어 선택과 정보·사실은 기준표로 다시 채점합니다(G-Eval 방식). 일상 모드는 채점하지 않습니다.

1. 항목 하나마다 프롬프트를 따로 만듭니다. 상황, 질문, 표준어 문장, 그리고 1~5점 각각의 설명이 들어갑니다. "답변의 길이와 말투(사투리 여부)는 이 항목에서 평가하지 않는다. 숫자 하나로만 답한다"는 지시를 붙입니다.
2. 기본 Gemma(어댑터 끔)가 답의 **첫 토큰**으로 "1"~"5"를 낼 확률을 읽습니다. 문장을 생성하지는 않습니다.
3. 확률로 기댓값을 내서 0~100점으로 바꿉니다.

점수 = (Σ k × P(k) − 1) ÷ 4 × 100 (k = 1~5)
예: P = (0.05, 0.10, 0.20, 0.50, 0.15) → 기댓값 3.6 → **65점**
다섯 숫자의 확률 합이 0.5 미만이면(숫자로 답하지 않음) 코치가 쓴 숫자를 그대로 씁니다.

항목	무엇을 보나	언제 채점하나
단어 선택	공식적인 자리에 맞는 정확한 어휘와 높임말	항상 (일상 모드 제외)
정보·사실	질문에 대한 구체적이고 정확한 정보와 근거	코치가 SCORE에 정보 점수를 적었을 때만
전달력	Gemma가 녹음을 직접 듣고 채점	**꺼짐**. 측정한 유창성과 상관이 r = −0.17(n = 12)이라 노이즈로 판단

12. 최종 점수

`server/analysis.py answer_score, final_score, script_coverage`

12.1 답변 점수

유창성은 녹음에서 잰 값(8장)을 쓰고, 잴 수 없을 때만 코치가 쓴 숫자를 씁니다.

경우	답변 점수
발표 모드 + 대본 있음	단어 선택 0.25 + 유창성 0.25 + **대본 일치도** 0.5
정보·사실 점수 있음	단어 선택 0.25 + 유창성 0.25 + 정보·사실 0.5
정보·사실 점수 없음	단어 선택 0.5 + 유창성 0.5
단어 선택이나 유창성이 없음	답변 점수 없음 → 표준어 점수만으로 판정

대본 일치도

대본 문장마다 "말했나"를 0~1로 냅니다. 두 값 중 큰 쪽을 씁니다.

- BERTScore 재현율(klue/roberta-base 8층, 0.667~0.74를 0~1로): 바꿔 말한 문장도 잡습니다.
- 글자 바이그램 재현율(0.30~0.60을 0~1로): 받아쓰기 오류가 있어도 원문 그대로 읽은 문장을 잡습니다.

문장 길이로 가중 평균해 0~100점을 내고, 0.5 미만인 문장은 "빠뜨린 문장"으로 표시합니다. 이 기준이면 소리 내 읽은 문장의 97%, 많이 바꿔 말한 문장의 70%를 말한 것으로 보고, 실제로 빠뜨린 문장은 3%만 말한 것으로 잘못 봅니다.

12.2 합격 판정

최종 점수 = 표준어 점수 + 답변 점수 (200점 만점) → **140점 이상 합격**, 화면에는 ÷ 2 해서 100점으로 표시
답변 점수가 없으면: 표준어 점수 × 2, **표준어 점수 70점 이상 합격**
일상 모드: 점수와 합격 판정 없음

예시 이어서 (9.4의 답변)

표준어 점수	단어 선택	유창성(측정)	정보·사실	답변 점수	최종
12	80	72	75	20 + 18 + 37.5 ≈ 76	**88 / 200 → 불합격** (화면 44점)

답변 내용이 괜찮아도 억양 때문에 표준어 점수가 12점이 되면 합격하기 어렵습니다. 부록 B의 첫 항목을 참고하세요.

부록 A. Gemma 어댑터 교체 기준

`server/engine.py _update (GATE_TOLERANCE, GATE_CONFIDENCE)`

엔진은 2시간마다 감시 목록(`SEOTTURI_WATCH`)에 새 체크포인트가 있는지 봅니다. 있으면 지금 어댑터와 함께 고정 벤치마크 160문장으로 채점하고, 아래 네 조건을 **모두** 통과할 때만 바꿉니다.

조건	기준
형식 준수율	지금 어댑터 이상
표준어 줄 CER	지금 어댑터 이하 (허용 오차 0, 10/7 사용자 결정)
방언 줄 CER	지금 어댑터 이하 (허용 오차 0)
짝지은 부트스트랩	두 줄 합산 CER이 더 나을 확률 ≥ 0.95

두 어댑터는 같은 디코딩(guard1)으로 채점한 점수끼리만 비교합니다. 실제로는 160문장으로 0.005 정도의 작은 개선을 가려내지 못해서, 최근 두 번은 수동으로 교체했습니다.

날짜	교체	근거
10/11	lora_v1 → lora_r64c	사용자 결정. 자동 판정은 나을 확률 0.67(통과 못 함), data_test 4,000문장과 data_all 검증 1,199문장에서 더 나음
10/11	lora_r64c → **lora_s6c**	사용자 결정. data_test 4,000문장(사이트와 같은 디코딩): 표준어 0.151 → 0.147, 방언 0.145 → 0.140, 나을 확률 1.00

지금 감시 목록에는 학습이 끝난 lora_r64c와 lora_s6c만 있어서 자동 판정을 기다리는 후보는 없습니다. Whisper 억양 모델은 자동 교체가 없고, `accent-id/runs/served` 링크를 바꾼 뒤 `./run.sh restart-accent`로 수동 교체합니다.

부록 B. 알아둘 점과 한계

1. 사투리 비율은 사실상 Whisper가 정합니다.
억양 판정이 나면 Gemma가 바꾸지 않은 어절 대부분이 억양 어절로 들어갑니다. 9.4 예시에서 단어만 비율은 27%인데 사투리 비율은 88%였습니다.

2. 표준 화자가 사투리로 나올 수 있습니다.
답변 단위 두 단계 판정에서 서울·경기 화자의 약 14~17%가 지역 억양으로 판정됩니다(화자 약 100명, 10/12). 조각 단위로는 시험 세트 서울 화자의 23%입니다. 이렇게 판정되면 표준어 단어만 써도 사투리 비율이 높게 나옵니다.

3. "두 줄의 차이"라는 판정 방식의 한계.
Gemma가 표준어 줄을 쓰면서 사투리와 관계없이 어절을 바꾸면(다듬기, 띄어쓰기를 넘는 수정, 한쪽 줄만 오인식) 그 어절도 사투리로 셉니다. 반대로 형식이 깨져 두 줄이 같아지면 그 조각의 사투리 단어는 0개가 됩니다.

4. 지역별 정확도 차이. lora_s6c의 방언 줄 CER(data_test, 사이트와 같은 디코딩)은 제주 0.295, 강원 0.151, 충청 0.096, 경상 0.071입니다. 제주는 받아쓰기가 덜 정확해서 사투리 단어 판정도 덜 믿을 만합니다. 전라는 Gemma 학습(라벨 문제로 제외)과 Whisper 학습 모두에 없습니다.

> **5. 억양 서비스가 꺼졌을 때.** 지역을 표시하지 않고, 사투리 비율은 Gemma 사투리 단어만으로 계산합니다. 같은 사람이라도 서비스 상태에 따라 점수가 크게 달라질 수 있습니다.

> **6. 코치는 녹음을 듣지 않습니다.** 발음·억양·속도에 대한 피드백은 모두 앞 단계 측정값을 글로 받은 것입니다. 녹음을 듣고 채점하는 전달력은 검증에서 맞지 않아 꺼 두었습니다.

부록 C. 코드 위치

단계	파일 · 함수
브라우저 문장 자르기	web/js/recorder.js
서버 전처리 · 조각 처리 전체	server/app.py transcribe · server/analysis.py preprocess, split_audio
Gemma 받아쓰기 · 디코딩 가드	server/engine.py _asr, _generate_ids, _finish, answer_budget, unloop
Gemma 프롬프트 · 출력 파싱	gemma4-dialect/common.py PROMPTS, prompt_inputs, parse_answer
Whisper 억양	server/accent.py load, classify · server/app.py accent_probs, zone_reliable
사투리 단어 판정	server/analysis.py diff
발음 검사	server/pron.py score, summary · server/analysis.py pron_summary
유창성	server/analysis.py fluency_clip, fluency_summary
지역 판정 · 사투리 비율	server/analysis.py summarize_speech, zone_verdict, accent_any_zone
코치 피드백	server/app.py coach_feedback · server/analysis.py feedback_messages
기준표 채점	server/app.py rubric_scores · server/analysis.py RUBRICS · server/engine.py _score
최종 점수 · 대본 일치도	server/analysis.py answer_score, final_score, script_coverage
어댑터 교체	server/engine.py _update, _compare · state/engine_state.json

경로는 /data/seotturi/seotturi-web 기준이고, gemma4-dialect는 /data/seotturi/gemma4-dialect, accent-id는 /data/seotturi/accent-id입니다.